

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ
ФАКУЛТЕТ КОМПЮТЪРНИ СИСТЕМИ И ТЕХНОЛОГИИ



КУРСОВ ПРОЕКТ

*Дисциплина: „Програмиране за мобилни
устройства“*

Тема: Приложение за прогноза за времето

Изготвил:

Йоан Димитров Ушагелов

Фак. №: 121223163

Група: 39

III курс, ВСИИ

София, 2026 г.

Съдържание

Увод.....	3
Анализ на съществуващи разработки	4
Google Weather (Времето от Google).....	4
AccuWeather.....	4
Weather & Radar	5
Сравнителен анализ и изводи	5
Проектиране	5
Целева група потребители	5
Използвани данни и структури.....	6
Основни data класове:.....	6
Потребителски интерфейс и навигация.....	6
Компоненти на главния екран (отгоре надолу):	7
Цветови схеми по вид на времето:	7
Архитектурна схема — MVVM.....	7
Реализация	8
Технологичен стек	8
REST API интеграция.....	8
MVVM Архитектура.....	9
GPS Локация.....	10
Push Нотификации	10
Обработка на грешки.....	11
Потребителско ръководство	11
Стартиране на приложението	11
Търсене на град	11
Използване на GPS локация.....	12
Разгъване на детайли за ден.....	12
Смяна на мерна единица	12
Ръчно обновяване.....	12
Ежедневни нотификации	12
Заклучение.....	13
Литература.....	14
Приложение — Програмен код	15
Data класове — WeatherModels.kt	15
Обработка на изключения, WeatherRepository.kt.....	15
DailyForecastAdapter с DiffUtil.....	16
Архитектурни слоеве.....	16

Увод

Прогнозата за времето е една от най-търсените услуги в ежедневието на съвременния човек. С нарастването на броя на смартфон потребителите и развитието на мобилните технологии, достъпът до актуална метеорологична информация е станал неотделна част от ежедневните навици на милиони хора по света.

Настоящият проект представя разработката на мобилно приложение за прогноза за времето, предназначено за платформата Android. Приложението е насочено към активни млади хора на възраст между 18 и 30 години, които желаят бързо и лесно да проверяват метеорологичните условия за текущия ден и за следващите 5 дни.

Основните проблеми, които приложението решава, са:

- Бавен и неудобен достъп до актуална информация за времето
- Липса на персонализация, потребителите не могат да запазват любими градове
- Неинтуитивни интерфейси на съществуващите решения
- Липса на автоматично определяне на местоположението
- Липса на ежедневни напомняния за прогнозата

Приложението е разработено с използването на модерни технологии: Kotlin, MVVM архитектура, Retrofit за REST API комуникация и WorkManager за планиране на нотификации. Интерфейсът следва принципите на Material Design и е изцяло на Български език.

Анализ на съществуващи разработки

При разработката на приложението са анализирани три от най-популярните мобилни приложения за прогноза за времето, достъпни в Google Play Store.

Google Weather (Времето от Google)

Google Weather е интегрирано директно в търсачката на Google и в приложението Google. Предлага изчистен и модерен интерфейс с почасова и дневна прогноза.

Характеристика	Оценка
Интерфейс	Изключително изчистен, минималистичен дизайн
Локация	Автоматично определяне чрез GPS
Прогноза	Почасова и 10-дневна прогноза
Нотификации	Не предлага ежедневни нотификации
Езикова поддръжка	Поддържа Български
Недостатъци	Не може да се използва самостоятелно като приложение

AccuWeather

AccuWeather е едно от най-популярните независими приложения за времето в света. Предлага подробна прогноза с много допълнителни метеорологични показатели.

Характеристика	Оценка
Интерфейс	Богат на информация, но претрупан
Прогноза	До 90-дневна прогноза
Нотификации	Богати нотификации с предупреждения
Реклами	Агресивна реклама в безплатната версия
Производителност	Бавно зареждане поради много данни
Недостатъци	Изисква платен абонамент за пълен достъп

Weather & Radar

Weather & Radar е популярно приложение с акцент върху интерактивни радарни карти и визуализации на метеорологичните явления.

Характеристика	Оценка
Интерфейс	Визуално атрактивен, модерен дизайн
Радар карти	Интерактивни карти в реално време
Нотификации	Предупреждения при екстремно време
Потребление	Висока консумация на батерия
Недостатъци	Сложен за начинаещи потребители

Сравнителен анализ и изводи

Анализът на съществуващите решения показва следните области за подобрение, които са адресирани в разработеното приложение:

- Интуитивен интерфейс, без излишна сложност, фокус върху основната информация
- Ежедневни нотификации — липсват в повечето безплатни решения
- Смяна на мерни единици °C/°F — не всички приложения предлагат лесна смяна
- Запазване на последния градус, персонализация без необходимост от акаунт
- Напълно безплатно — без реклами и абонамент

Проектиране

Целева група потребители

Приложението е насочено към следната целева група:

Възраст	18 — 30 години
Платформа	Android смартфони (API 24+, Android 7.0+)
Нужди	Бърза проверка на времето, без излишна сложност
Технически умения	Средно ниво — запознати с мобилни приложения
Употреба	Ежедневна, при планиране на деня
Очаквания	Бързо зареждане, ясен интерфейс, точни данни

Използвани данни и структури

Приложението използва данни от OpenWeatherMap API (<https://openweathermap.org/api>). API-то връща JSON данни, които се десериализират в Kotlin data класове.

Основни data класове:

Клас	Описание	Полета
WeatherResponse	Текущо време	name, main, weather, wind, visibility, sys, dt
Main	Температурни данни	temp, feelsLike, tempMin, tempMax, humidity, pressure
WeatherDescription	Описание	id, main, description, icon
Wind	Вятър	speed, deg
ForecastResponse	5-дневна прогноза	list: List<ForecastItem>, city
ForecastItem	Единична прогноза	dt, main, weather, wind, precipProbability, dtTxt
CurrentWeatherUi	UI модел	cityName, tempC, description, humidity, windKmh...
DayForecastUi	UI модел за ден	dayName, iconCode, tempHighC, tempLowC, humidity...
HourForecastUi	UI модел за час	timeLabel, iconCode, tempC

Данните се запазват локално чрез SharedPreferences:

- Последно търсен/избран град (String)
- Предпочитана мерна единица °C или °F (Boolean)
- Настройка за нотификации (Boolean)

Потребителски интерфейс и навигация

Приложението се състои от един главен екран (MainActivity) с вертикален scroll. Дизайнът следва Material Design принципите с динамични цветови схеми.

Компоненти на главния екран (отгоре надолу):

- Лента за търсене с бутони: Търси, GPS локация, Обнови
- Текуща температура
- Град и държава, описание
- Карта със статистики: Влажност, Вятър, Видимост, Изгрев/Залез
- Хоризонтален RecyclerView с почасова прогноза
- Вертикален RecyclerView с 5-дневна прогноза (разгъваеми редове)
- Бутони за смяна на мерна единица °C / °F

Цветови схеми по вид на времето:

Вид на времето	Икон код	Начален цвят	Краен цвят
Слънчево	01d	#1565C0	#42A5F5
Частично облачно	02d, 03d	#455A64	#90A4AE
Облачно	04d	#37474F	#78909C
Дъжд	09d, 10d	#1A237E	#5C6BC0
Буря	11d	#212121	#546E7A
Сняг	13d	#B3E5FC	#81D4FA
Мъгла	50d	#607D8B	#B0BEC5
Нощ	*n суфикс	#0D1B2A	#1B2A4A

Архитектурна схема — MVVM

Приложението следва MVVM (Model-View-ViewModel) архитектура с Repository pattern:

Слой	Класове	Отговорност
View (UI)	MainActivity, HourlyForecastAdapter, DailyForecastAdapter	Показване на данни, взаимодействие с потребителя
ViewModel	WeatherViewModel	Бизнес логика, LiveData, управление на състоянието
Repository	WeatherRepository	Координира данните между API и UI
Network	WeatherApiService, RetrofitClient	REST API заявки към OpenWeatherMap
Local	WeatherPreferences	Локално запазване с SharedPreferences
Worker	WeatherNotificationWorker	Фоновы задачи за нотификации

Реализация

Технологичен стек

Технология	Версия	Предназначение
Kotlin	1.8.22	Основен език за програмиране
Android SDK	API 34 (Android 14)	Целева платформа
Retrofit2	2.9.0	HTTP клиент за REST API
OkHttp3	4.11.0	HTTP комуникация и логване
Gson	—	JSON десериализация
Coroutines	1.7.3	Асинхронно програмиране
ViewModel + LiveData	2.7.0	MVVM архитектура
Glide	4.16.0	Зареждане на изображения
WorkManager	2.9.0	Планиране на нотификации
FusedLocationProvider	21.1.0	GPS локация
Material Components	1.11.0	Material Design UI
ViewBinding	—	Типово-безопасен достъп до layout елементи

REST API интеграция

Комуникацията с OpenWeatherMap API се осъществява чрез Retrofit2 с OkHttp клиент. Дефиниран е интерфейс с suspend функции за асинхронни заявки:

```
// WeatherApiService.kt
@GET("weather")
suspend fun getCurrentWeatherByCity(
    @Query("q") cityName: String,
    @Query("appid") apiKey: String,
    @Query("units") units: String = "metric",
    @Query("lang") lang: String = "bg"
): Response<WeatherResponse>
```

Параметърът lang="bg" осигурява получаването на описания директно на Български от API-то. Параметърът units="metric" връща температурата в Celsius.

RetrofitClient е имплементиран като Kotlin object (Singleton) за осигуряване на единствена инстанция:


```
// RetrofitClient.kt
object RetrofitClient {
    private const val BASE_URL =
"https://api.openweathermap.org/data/2.5/"
    val weatherApiService: WeatherApiService by lazy {
        retrofit.create(WeatherApiService::class.java)
    }
}
```

MVVM Архитектура

WeatherViewModel наследява ViewModel и използва viewModelScope за стартиране на coroutines. Данните се излагат чрез LiveData обекти, наблюдавани от MainActivity:

```
// WeatherViewModel.kt
fun loadWeatherByCity(cityName: String) {
    _isLoading.value = true
    viewModelScope.launch {
        val result = repository.getCurrentWeather(cityName)
        result.onSuccess { _currentWeather.value = it }
            .onFailure { _errorMessage.value = it.message }
        _isLoading.value = false
    }
}
```

WeatherRepository използва Kotlin Result тип за обработка на успешни и неуспешни резултати, без да хвърля изключения към UI слоя:

```
// WeatherRepository.kt
suspend fun getCurrentWeather(cityName: String): Result<CurrentWeatherUi>
{
    return try {
        val response = api.getCurrentWeatherByCity(cityName, apiKey)
        if (response.isSuccessful) {
            Result.success(mapToCurrentUi(response.body()!!))
        } else {
            when (response.code()) {
                404 -> Result.failure(Exception("Градът не е намерен"))
                401 -> Result.failure(Exception("Невалиден API ключ"))
                else -> Result.failure(Exception("Грешка:
${response.code()}"))
            }
        }
    } catch (e: Exception) {
        Result.failure(Exception("Няма интернет връзка"))
    }
}
```

GPS Локация

Достъпът до GPS координатите се осъществява чрез FusedLocationProviderClient от Google Play Services. Правата за локация се заявяват динамично чрез ActivityResultContracts:

```
// MainActivity.kt
private val locationPermissionRequest = registerForActivityResult(
    ActivityResultContracts.RequestMultiplePermissions()
) { permissions ->
    if (permissions[ACCESS_FINE_LOCATION] == true) {
        getDeviceLocation()
    }
}
```

```
@SuppressLint("MissingPermission")
private fun getDeviceLocation() {
    fusedLocationClient.getCurrentLocation(
        Priority.PRIORITY_HIGH_ACCURACY, null
    ).addOnSuccessListener { location ->
        viewModel.loadWeatherByLocation(location.latitude,
            location.longitude)
    }
}
```

Push Нотификации

Ежедневните нотификации са реализирани чрез WorkManager, което гарантира тяхното изпълнение дори ако приложението е затворено. Нотификацията се планира за 08:00 сутринта:

```
// NotificationHelper.kt
fun scheduleDaily(context: Context) {
    val now = Calendar.getInstance()
    val target = Calendar.getInstance().apply {
        set(Calendar.HOUR_OF_DAY, 8)
        set(Calendar.MINUTE, 0)
        if (before(now)) add(Calendar.DAY_OF_YEAR, 1)
    }
    val initialDelay = target.timeInMillis - now.timeInMillis
    val workRequest =
        PeriodicWorkRequestBuilder<WeatherNotificationWorker>(24, HOURS)
            .setInitialDelay(initialDelay, MILLISECONDS)
            .setConstraints(Constraints.Builder()
                .setRequiredNetworkType(NetworkType.CONNECTED).build())
            .build()
    WorkManager.getInstance(context).enqueueUniquePeriodicWork(
        "weather_daily", ExistingPeriodicWorkPolicy.KEEP, workRequest
    )
}
```

Обработка на грешки

Приложението обработва всички видове грешки по елегантен начин, без да блокира UI нишката:

Вид грешка	Обработка	Потребителско съобщение
Няма интернет	catch(Exception)	"Няма интернет връзка"
Невалиден град	HTTP 404	"Градът не е намерен"
Невалиден API ключ	HTTP 401	"Невалиден API ключ"
GPS отказан	onFailure callback	Toast съобщение
Локация недостъпна	Null check	Toast съобщение
Сървърна грешка	HTTP 5xx	"Грешка: {код}"

Потребителско ръководство

Това ръководство описва как потребителят взаимодейства с приложението.

Стартиране на приложението

При стартиране приложението автоматично зарежда времето за последно избрания град (по подразбиране: София). Показват се:

- Икона на текущото метеорологично явление
- Текуща температура в голям формат
- Описание (Слънчево, Облачно, Дъжд и др.)
- Статистики — влажност, вятър, видимост
- Прогноза по часове и 5-дневна прогноза

Търсене на град

- Натиснете полето за търсене горе на екрана
- Въведете името на град
- Натиснете иконката за търсене
- Приложението зарежда данните за новия град и го запазва

Използване на GPS локация

- Натиснете иконката за локация () горе вдясно
- При първо използване позволете достъп до локацията
- Приложението определя координатите и зарежда времето за текущото местоположение

Разгъване на детайли за ден

- В секцията „5-дневна прогноза“ натиснете върху желания ден
- Редът показва допълнителна информация
 - Влажност и вятър за деня
 - Вероятност за валеж
 - Час на изгрев и залез
- Натиснете отново за скриване на детайлите

Смяна на мерна единица

В долната част на екрана са бутоните °C и °F. Натиснете желаната единица всички температури на екрана се актуализират мигновено. Изборът се запазва при следващо стартиране.

Ръчно обновяване

Натиснете иконката за обновяване () горе вдясно. Данните се зареждат наново от API-то. При успешно обновяване иконката се завърта 360 градуса.

Ежедневни нотификации

Всеки ден в 08:00 сутринта приложението изпраща нотификация с текущата прогноза за последно избрания град. Нотификацията показва града, температурата и описанието на времето. За да работят нотификациите, е необходима активна интернет връзка.

Заклучение

Разработеното мобилно приложение за прогноза за времето успешно изпълнява всички поставени изисквания и цели. Приложението предоставя:

- Актуална метеорологична информация от реален REST API (OpenWeatherMap)
- Интуитивен интерфейс на Български език с Material Design
- Функционалности на устройството — GPS локация и Push нотификации
- Чиста MVVM архитектура с Repository pattern
- Надеждна обработка на грешки за всички сценарии
- Персистентност — запазване на последния град и предпочитания

Сравнено с анализираните съществуващи решения, приложението предлага баланс между богата функционалност и простота на употреба. Липсата на реклами и изискването за акаунт го правят по-достъпно за целевата аудитория.

Приложението е готово за употреба и може да бъде публикувано в Google Play Store след добавяне на собствен API ключ. Бъдещи разширения могат да включват: уиджет за началния екран, интерактивни радарни карти, поддръжка на множество градове и тъмен режим.

Литература

- Android Developers Documentation. (2024). Guide to app architecture.
<https://developer.android.com/topic/architecture>
- Android Developers Documentation. (2024). ViewModel overview.
<https://developer.android.com/topic/libraries/architecture/viewmodel>
- OpenWeatherMap. (2024). Current weather data API.
<https://openweathermap.org/current>
- Square Inc. (2024). Retrofit — A type-safe HTTP client for Android.
<https://square.github.io/retrofit/>
- Google. (2024). Material Design 3. <https://m3.material.io/>
- JetBrains. (2024). Kotlin Coroutines Guide.
<https://kotlinlang.org/docs/coroutines-guide.html>
- Android Developers. (2024). WorkManager — Schedule tasks.
<https://developer.android.com/topic/libraries/architecture/workmanager>
- Google. (2024). Fused Location Provider API.
<https://developers.google.com/location-context/fused-location-provider>
- Bumptech. (2024). Glide — Image Loading Library.
<https://bumptech.github.io/glide/>
- Google. (2024). ViewBinding.
<https://developer.android.com/topic/libraries/view-binding>

Приложение — Програмен код

В тази секция са представени най-важните части от програмния код на приложението.

Data класове — WeatherModels.kt

```
data class WeatherResponse(  
    val name: String,  
    val main: Main,  
    val weather: List<WeatherDescription>,  
    val wind: Wind,  
    val visibility: Int,  
    val sys: Sys,  
    val dt: Long  
)
```

```
data class Main(  
    val temp: Double,  
    @SerializedName("feels_like") val feelsLike: Double,  
    val humidity: Int,  
    val pressure: Int  
)
```

```
data class CurrentWeatherUi(  
    val cityName: String,  
    val tempC: Int,  
    val description: String,  
    val iconCode: String,  
    val humidity: Int,  
    val windKmh: Int  
) {  
    fun tempDisplay(isCelsius: Boolean): String {  
        val t = if (isCelsius) tempC else (tempC * 9.0 / 5 + 32).toInt()  
        return "$t°${if (isCelsius) "C" else "F"}"  
    }  
}
```

Обработка на изключения, WeatherRepository.kt

```
suspend fun getCurrentWeather(cityName: String): Result<CurrentWeatherUi>  
{  
    return try {  
        val response = api.getCurrentWeatherByCity(cityName, apiKey)  
        if (response.isSuccessful) {  
            val body = response.body()  
            ?: return Result.failure(Exception("Празен отговор"))  
            Result.success(mapToCurrentUi(body))  
        }  
    }  
}
```

```

        } else {
            when (response.code()) {
                404 -> Result.failure(Exception("Градът не е намерен"))
                401 -> Result.failure(Exception("Невалиден API ключ"))
                else -> Result.failure(Exception("Грешка:
${response.code()}"))
            }
        }
    } catch (e: Exception) {
        Result.failure(Exception("Няма интернет връзка"))
    }
}

```

DailyForecastAdapter с DiffUtil

```

class DailyForecastAdapter(private var isCelsius: Boolean)
    : ListAdapter<DayForecastUi,
DailyForecastAdapter.ViewHolder>(DiffCallback) {

```

```

    companion object DiffCallback :
DiffUtil.ItemCallback<DayForecastUi>() {
        override fun areItemsTheSame(old: DayForecastUi, new:
DayForecastUi) =
            old.dayName == new.dayName
        override fun areContentsTheSame(old: DayForecastUi, new:
DayForecastUi) =
            old == new
    }
}

```

Архитектурни слоеве

Файл	Слой	Паттерн
MainActivity.kt	View	Observer (наблюдава LiveData)
WeatherViewModel.kt	ViewModel	MVVM ViewModel
WeatherRepository.kt	Repository	Repository Pattern
RetrofitClient.kt	Network	Singleton Pattern
WeatherApiService.kt	Network	Interface / Retrofit
WeatherPreferences.kt	Local	Wrapper / Façade
WeatherNotificationWorker.kt	Worker	WorkManager Worker
HourlyForecastAdapter.kt	View	Adapter Pattern (RecyclerView)
DailyForecastAdapter.kt	View	Adapter Pattern + DiffUtil